

Tuples and Dictionaries in Python

Asma Khan

Python Collections (Arrays)

- There are four collection data types in the Python programming language:
- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered and unindexed. No duplicate members.
- **Dictionary** is a collection which is unordered, changeable and indexed. No duplicate members.

Tuple

A **Tuple** is a collection of **Python** objects separated by commas. In some ways a **tuple** is similar to a list in terms of indexing, nested objects and repetition but a **tuple** is immutable unlike lists which are mutable.

```
# An empty tuple
```

```
tuple1=()
```

```
print(tuple1)
```

```
tup2=('Python', 'C++', 'Java')  
print(tup2)
```

#Concatenation of two tuples

```
tup1=(1,2,3,4)  
tup2=('Apple','grapes','Mango')  
tup3=tup1+tup2  
print(tup1+tup2)
```

- # Code for creating nested tuples

- tuple1 = (0, 1, 2, 3)

- tuple2 = ('python', 'geek')

- tuple3 = (tuple1, tuple2)

- print(tuple3)

Output:

((0, 1, 2, 3), ('python', 'geek'))

- #code to test that tuples are immutable
- tuple1 = (0, 1, 2, 3)
- tuple1[0] = 4
- print(tuple1)

- Would generate error.

code to test slicing

- tuple1 = (0 ,1, 2, 3)
- print(tuple1[1:])
- print(tuple1[::-1])
- print(tuple1[2:4])

Output:

(1, 2, 3) (3, 2, 1, 0) (2, 3)

- `# Code for printing the length of a tuple`
- `tuple2 = ('python', 'geek')`
- `print(len(tuple2))`

- `# Code for converting a list and a string into a tuple`
- `list1 = [0, 1, 2]`
- `print(tuple(list1))`
- `print(tuple('python')) # string 'python'`
- `(0, 1, 2) ('p', 'y', 't', 'h', 'o', 'n')`

#python code for creating tuples in a loop

- ```
thistuple = ("apple", "banana", "cherry")
for x in thistuple:
 print(x)
```

#Check if item exists

Check if "apple" is present in the tuple:

```
thistuple = ("apple", "banana", "cherry")
if "apple" in thistuple:
 print("Yes, 'apple' is in the fruits tuple")
```

Once a tuple is created, you cannot change its values. Tuples are **unchangeable**, or **immutable** as it also is called.

But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

- ```
x = ("apple", "banana", "cherry")  
y = list(x)  
y[1] = "kiwi"  
x = tuple(y)  
  
print(x)
```

Adding Set elements

Add Items

To add one item to a set use the `add()` method.

To add more than one item to a set use the `update()` method.

Example

Add an item to a set, using the `add()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.add("orange")
```

```
print(thisset)
```

Remove Item

To remove an item in a set, use the `remove()`, or the `discard()` method.

Example

Remove "banana" by using the `remove()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.remove("banana")
```

```
print(thisset)
```

Remove "banana" by using the `discard()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.discard("banana")
```

```
print(thisset)
```

Add multiple items to a set, using the `update()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.update(["orange", "mango", "grapes"])
```

```
print(thisset)
```

Using Pop and Clear methods

Remove the last item by using the `pop()` method:

```
thisset = {"apple", "banana", "cherry"}  
x = thisset.pop()
```

```
print(x)
```

```
print(thisset)
```

The `clear()` method empties the set:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.clear()
```

```
print(thisset)
```

Join Two Sets

There are several ways to join two or more sets in Python.

You can use the `union()` method that returns a new set containing all items from both sets, or the `update()` method that inserts all the items from one set into another:

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1.union(set2)
```

```
print(set3)
```

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set1.update(set2)
```

```
print(set1)
```

```
fruits =
```

- `x = {"apple", "banana", "cherry"}`
- `y = {"google", "microsoft", "apple"}`
- `z = x.difference(y)`
- `print(z)`

```
Fruits={"apple", "banana", "cherry"}  
if ("apple" in fruits):  
    print("Yes, apple is a fruit!")
```


Python Dictionary

Dictionary in Python is an unordered collection of data values, used to store data values like a map, which unlike other Data Types that hold only single value as an element, Dictionary holds **key:value** pair.

Key value is provided in the dictionary to make it more optimized.

Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.

In Python, a Dictionary can be created by placing sequence of elements within curly {} braces, separated by 'comma'.

Dictionary holds a pair of values, one being the Key and the other corresponding pair element being its **Key:value**.

Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be *immutable*.

Dictionary can also be created by the built-in function dict(). An empty dictionary can be created by just placing curly braces {}.

.

- # Creating an empty Dictionary
 - Dict = {}
 - print("Empty Dictionary: ")
 - print(Dict)
-
- # Creating a Dictionary
 - # with Integer Keys
 - Dict = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
 - print("\nDictionary with the use of Integer Keys: ")
 - print(Dict)
-
- # Creating a Dictionary
 - # with Mixed keys
 - Dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}
 - print("\nDictionary with the use of Mixed Keys: ")

- # Creating a Dictionary
- # with dict() method
- Dict = dict({1: 'Geeks', 2: 'For', 3:'Geeks'})
- print("\nDictionary with the use of dict(): ")
- print(Dict)

- # Creating a Dictionary
- # with each item as a Pair
- Dict = dict([(1, 'Geeks'), (2, 'For')])
- print("\nDictionary with each item as a pair: ")
- print(Dict)

Adding elements to a Dictionary

In Python Dictionary, Addition of elements can be done in multiple ways.

One value at a time can be added to a Dictionary by defining value along with the key e.g. `Dict[Key] = 'Value'`. Updating an existing value in a Dictionary can be done by using the built-in **update()** method.

Nested key values can also be added to an existing Dictionary.

Note- While adding a value, if the key value already exists, the value gets updated otherwise a new Key with the value is added to the Dictionary.

- # Creating an empty Dictionary

- Dict = {}

- print("Empty Dictionary: ")

- print(Dict)

- # Adding elements one at a time

- Dict[0] = 'Geeks'

- Dict[2] = 'For'

- Dict[3] = 1

- print("\nDictionary after adding 3 elements: ")

- print(Dict)

- # Adding set of values
- # to a single Key
- Dict['Value_set'] = 2, 3, 4
- print("\nDictionary after adding 3 elements: ")
- print(Dict)

- # Updating existing Key's Value
- Dict[2] = 'Welcome'
- print("\nUpdated key value: ") b
- print(Dict)

- # Adding Nested Key value to Dictionary
- Dict[5] = {'Nested' :{'1' : 'Life', '2' : 'Geeks'}}
- print("\nAdding a Nested Key: ")
- print(Dict)

Python program to demonstrate # accessing a element from a Dictionary

- # Creating a Dictionary
- Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

- # accessing a element using key
- print("Acessing a element using key:")
- print(Dict['name'])

- # accessing a element using key
- print("Acessing a element using key:")
- print(Dict[1])

- # accessing a element using get()
- # method
- print("Acessing a element using get:")
- print(Dict.get(3))

Removing elements from dictionary

- Dict = { 5 : 'Welcome', 6 : 'To', 7 : 'Geeks', 'A' : {1 : 'Geeks', 2 : 'For', 3 : 'Geeks'}, 'B' : {1 : 'Geeks', 2 : 'Life'}}
- print("Initial Dictionary: ")
- print(Dict)

- # Deleting a Key value
- del Dict[6]
- print("\nDeleting a specific key: ")
- print(Dict)

- # Deleting a Key from
- # Nested Dictionary
- del Dict['A'][2]
- print("\nDeleting a key from Nested Dictionary: ")
- print(Dict)

- # Deleting a Key
 - # using pop()
 - Dict.pop(5)
 - print("\nPopping specific element: ")
 - print(Dict)
-
- # Deleting an arbitrary Key-value pair
 - # using popitem()
 - Dict.popitem()
 - print("\nPops an arbitrary key-value pair: ")
 - print(Dict)
-
- # Deleting entire Dictionary
 - Dict.clear()
 - print("\nDeleting Entire Dictionary: ")